

Phase 4: System Analysis & Design

Project Title: AI-Based In-Cabin Monitoring System

1. Problem Statement & Architecture Strategy

Problem Statement

Current vehicle monitoring systems are predominantly reactive, recording video for post-incident review rather than proactively intervening. They also raise significant privacy concerns by streaming cabin footage to cloud servers. The proposed system solves this by providing a 100% offline, real-time edge-computing solution that detects driver fatigue, distraction, and in-cabin violence instantly, while ensuring authorized access via facial recognition.

Software Architecture Style

The system employs a **Modular Monolith Architecture** centered around a Python Flask backend.

- **Backend:** Flask handles the REST API, MJPEG video streaming, and dashboard metric polling.
 - **AI Processing:** Background threads run parallel inference pipelines (YOLOv8, YOLO11, ArcFace, MoViNet) utilizing PyTorch and Ultralytics, accelerated by CUDA.
 - **Data Storage:** Lightweight local storage using SQLite for event logs and JSON for user embeddings, fitting the edge-deployment constraint.
 - **Frontend:** A Vanilla HTML/CSS/JS dashboard decoupled from heavy frontend frameworks to minimize overhead and maximize responsiveness.
-

2. Database Design & Data Modeling

The system relies on local edge storage. The database architecture is split into structured relational logs (SQLite) and document-based biometric storage (JSON).

Logical Schema (SQLite: logs/system_logs.db)

Table: events

- event_id (Primary Key, Auto-increment, INT)
- timestamp (DATETIME, Default CURRENT_TIMESTAMP)
- module (VARCHAR) - e.g., 'Drowsiness', 'Distraction', 'Violence'
- alert_type (VARCHAR) - e.g., 'MICROSLEEP', 'WARNING', 'CRITICAL'
- duration_s (FLOAT) - Duration of the event in seconds
- confidence_score (FLOAT) - e.g., Violence probability or YOLO confidence
- notes (TEXT) - Additional metadata (e.g., "Phone detected", "EAR < 0.24")

Document Store (data/users_db.json)

Used for Face Authentication embeddings and metadata.

- user_id (UUID)
- name (String)
- arcface_embedding (Array of Floats)
- registered_at (Timestamp)

[PLACEHOLDER FOR ER DIAGRAM]

(Note to team: Insert an ER Diagram here showing a unified database schema if you decide to merge all logs into a single relational database).

3. Required System & UML Diagrams

Based on the university guidelines, you will need to generate the following diagrams. Below is the blueprint of what each diagram must depict for your specific system:

A. Data Flow & System Behavior

- **Context-Level DFD (Level 0):** Show the "Driver/Cabin" as the external entity sending "Live Video Feed" to the central "In-Cabin Monitoring System", which outputs "Audio/Visual Alerts" to the Driver and "Event Logs" to the Local Database.
- **Sequence Diagram (Login):** 1. Driver initiates login. 2. Camera captures frame. 3. YOLOv8m-face detects face. 4. Anti-spoofing module verifies liveness. 5. ArcFace extracts embedding. 6. System compares with users_db.json. 7. Access granted/denied.
- **State Diagram (Threat Evaluator):** Illustrate the transitions of the Violence/Weapon threat logic: NORMAL → WATCH (Suspicious/Weapon present) → WARNING (Violence > 0.6) → CRITICAL (Weapon + Violence > 0.6).
- **Activity Diagram (Drowsiness Pipeline):** Show the parallel split of a frame going to YOLO11n (bounding boxes) and MediaPipe (FaceMesh landmarks), merging at the "OR

Logic Gate", updating the PERCLOS buffer, and conditionally triggering Alerts.

B. Structural Diagrams

- **Class Diagram:** Define backend Python classes. Key classes should include FlaskServer, DistractionDetector, DrowsinessDetector, ViolenceDetector (inheriting from a base Detector class), and a DatabaseLogger class.
- **Component Diagram:** Visualize the physical structure: Webcam component → AI Models (YOLO, MoViNet, ArcFace) → Flask API → Dashboard UI & SQLite Database.
- **Deployment Diagram:** Show a single edge node (e.g., "In-Car Processing Unit"). Inside the node, depict the OS (Ubuntu/Windows), GPU (GTX 1650/1660), CUDA environment, and the running Python Virtual Environment housing the application.

4. Hardware & Technology Stack Baseline

Layer	Technologies Used
Core Languages	Python 3.11.5, JavaScript (ES6+), HTML5, CSS3
Deep Learning	PyTorch 2.5.1/2.2.1 (CUDA 12.1/11.8), Ultralytics (YOLOv8/11)
Computer Vision	OpenCV 4.13, MediaPipe ≥ 0.10, InsightFace (ArcFace)
Backend / Web	Flask 3.0.0, Flask-CORS, Chart.js (Frontend metrics)
Hardware	NVIDIA GTX 1650 / 1660 Super, 1280x720 PC Webcam